

Project Requirements

Electronic Health Record System on Blockchain

1. Project Overview

This project implements a decentralised Electronic Health Record (EHR) system built on Hyperledger Fabric blockchain with IPFS-based off-chain storage. The system manages the complete lifecycle of patient data — from registration and visits through lab testing, prescriptions, and insurance claim processing — across multiple organisations.

Key Technologies:

Component	Technology / Tool
Blockchain	Hyperledger Fabric (multi-org channel)
Off-chain Storage	IPFS via Kubo node (ipfs-service)
Backend APIs	Node.js / Express.js
Authentication	JWT (JSON Web Tokens) + bcrypt
Smart Contracts	Fabric Chaincode (Go/JS)

2. System Architecture

The system is composed of six microservices (API servers), each serving one or more roles/organisations on the Fabric network:

Service	Port	Organisation / Roles Served
ipfs-service	3006	Internal — IPFS wrapper for all services
peer0-api	3001	HospitalMSP — Receptionist, Hospital Admin
peer1-api	3002	HospitalMSP — Doctor
peer2-api	3003	HospitalMSP — Nurse, Pharmacist, Medical Record Officer
extorg-api	3004	DiagnosticsMSP + ProviderMSP — Lab & Claims roles
patient-api	3005	Patient self-service portal

3. Organisations and User Roles

3.1 HospitalMSP

Role	Peer	Responsibilities
receptionist	peer0	Registers patients, opens/closes visits
hospitaladmin	peer0	Administrative oversight of hospital operations
doctor	peer1	Manages diagnoses, prescriptions, lab requests, visit discharge
nurse	peer2	Records vitals and nursing notes
pharmacist	peer2	Dispenses medications, confirms prescriptions
medrecordofficer	peer2	Manages and archives medical records

3.2 DiagnosticsMSP

Role	Peer	Responsibilities
labreceptionist	peer0.diagnostic	Acknowledges incoming lab requests
labtechnician	peer0.diagnostic	Performs tests and submits results
radiologist	peer0.diagnostic	Handles imaging/radiology requests
labsupervisor	peer0.diagnostic	Approves lab results, returns to doctor
labadmin	peer0.diagnostic	Full lab management access

3.3 ProviderMSP

Role	Peer	Responsibilities
billingofficer	peer0.provider	Submits insurance claims for visits
claimsauditor	peer0.provider	Audits submitted claims with notes
insuranceofficer	peer0.provider	Approves or rejects claims
provideradmin	peer0.provider	Full provider/insurance admin access

4. Functional Requirements

4.1 Patient Registration and EHR Initialisation

- Receptionist registers a new patient on the blockchain (RegisterPatient).
- System automatically creates an empty EHR JSON template on IPFS.
- IPFS Content Identifier (CID) is stored on-chain via InitEHR chaincode.
- Each patient receives a unique Patient ID (e.g., PAT-001).

4.2 Visit Management

- Receptionist opens a visit for a patient (OpenVisit) with chief complaint details.
- A visit JSON is created on IPFS before the blockchain transaction.
- Doctor is assigned to the visit; doctor can update diagnosis notes.
- Nurse records vitals and nursing observations during the visit.
- Doctor closes (discharges) the visit upon completion of treatment.
- Receptionist can formally close the visit after discharge.

4.3 Prescription Management

- Doctor writes prescriptions and attaches them to the visit record on IPFS.
- Pharmacist views and dispenses prescribed medications.
- Pharmacist confirms dispensing, which updates the visit IPFS record.

4.4 Lab and Diagnostics Workflow

- Doctor raises a lab request (tests, notes) attached to the visit record.
- Lab Receptionist acknowledges the request (status: ACKNOWLEDGED).
- Lab Technician or Radiologist submits test results (status: COMPLETED).
- Lab Supervisor approves results and marks them ready (status: APPROVED).
- Lab results are returned to the treating doctor via ForwardContract.
- All lab actions are recorded in the visit's forwardingLog on IPFS.

4.5 Insurance Claims Processing

- Billing Officer submits an insurance claim for a completed visit (SubmitClaim).
- Claims Auditor reviews and annotates the claim with audit notes (AuditClaim).
- Insurance Officer processes the claim — decision: APPROVED or REJECTED (ProcessClaim).
- All claim events are recorded in the visit's forwardingLog on IPFS.

4.6 Patient Self-Service Portal

- Patient can log in and view their own EHR record.
- EHR CID is retrieved from blockchain; full record is fetched from IPFS.
- Patient can view visit history and associated clinical data.

4.7 Medical Record Management

- Medical Record Officer can archive and manage historical patient records.
- Full visit history is available via blockchain history queries.

5. Non-Functional Requirements

5.1 Security

- All API endpoints (except /health) require JWT bearer token authentication.
- Passwords are stored as bcrypt hashes; plaintext passwords are never stored.
- Role-based access control (RBAC) enforced at route level on every endpoint.
- IPFS service protected by a shared secret key (X-IPFS-Key header).
- Helmet.js and CORS middleware applied on all Express servers.
- Rate limiting: maximum 100 requests per minute per service.
- JWT tokens expire after 8 hours by default (configurable via .env).

5.2 Data Integrity

- Clinical data (EHR, visit JSON) is stored on IPFS; CIDs are stored on-chain.
- Any modification to off-chain data produces a new CID, recorded on blockchain.
- Full audit trail maintained via blockchain transaction history.
- forwardingLog in every visit JSON records all actor transitions and actions.
- Result hashes can be stored alongside lab results for integrity verification.

5.3 Performance and Scalability

- Request body size limited to 1 MB (APIs) and 5 MB (IPFS service).
- Peer routing is role-aware; each role connects to its designated Fabric peer.
- Services are independently deployable and restartable without full system downtime.

5.4 Availability and Reliability

- Health check endpoint (/health) available on every service without authentication.
- Graceful shutdown handling for SIGTERM and SIGINT signals.
- Structured logging (info, warn, error, http) with separate error log files.

6. IPFS Service API Requirements

The ipfs-service (port 3006) is a thin HTTP wrapper around a Kubo IPFS node. All EHR and Visit JSON objects are stored through this service.

Method	Endpoint	Purpose
GET	/health	IPFS node status check (no auth required)
POST	/pin	Pin any JSON object; returns CID
GET	/fetch/:cid	Fetch JSON content by CID
POST	/ehr/init	Create and pin empty EHR template for a patient
POST	/visit/init	Create and pin empty visit JSON for a new visit
POST	/unpin	Unpin a CID (history is preserved on blockchain)
GET	/pins	List all pinned CIDs (debug/admin use)

7. Lab API Endpoints (extorg-api)

Method	Endpoint	Role Required	Action
GET	/lab/visits/:id	Lab roles	View visit + clinical data
GET	/lab/visits/:id/request/:reqId	Lab roles	View specific lab request
PUT	/lab/visits/:id/request/:reqId/acknowledge	labreceptionist, labadmin	Acknowledge lab request
PUT	/lab/visits/:id/request/:reqId/submit	labtechnician, radiologist	Submit test results
PUT	/lab/visits/:id/request/:reqId/approve	labsupervisor, labadmin	Approve lab results

PUT	/lab/visits/:id/request/:reqId/return	labsupervisor, labadmin	Return results to doctor
-----	---------------------------------------	-------------------------	--------------------------

8. Claims API Endpoints (extorg-api)

Method	Endpoint	Role Required	Action
GET	/claims/visits/:id	All provider roles	View visit + clinical data
GET	/claims/visits/:id/history	All provider roles	View full visit history
POST	/claims/visits/:id/submit	billingofficer	Submit insurance claim
PUT	/claims/visits/:id/audit	claimsauditor	Audit claim with notes
PUT	/claims/visits/:id/process	insuranceofficer	Approve or reject claim

9. Authentication API Endpoints

Method	Endpoint	Description
POST	/auth/login	Accepts username + password, returns JWT token and user info
GET	/auth/me	Returns current authenticated user details (requires token)

Login response includes: JWT token, expiry duration (default 8h), and user metadata (username, role, mspld, peer). All subsequent requests must include Authorization: Bearer <token> header.

10. Environment Configuration Requirements

Required .env variables per service:

Variable	Description
PORT	Port number the service listens on
JWT_SECRET	Secret key for signing JWT tokens
JWT_EXPIRES_IN	Token expiry duration (e.g., 8h)
FABRIC_BASE_PATH	Path to Fabric network configuration files
DIAG_PEER_ADDRESS	DiagnosticsMSP peer address (extorg-api)
PROV_PEER_ADDRESS	ProviderMSP peer address (extorg-api)
IPFS_API_URL	Kubo IPFS API URL (default: http://localhost:5001)
IPFS_GATEWAY_URL	IPFS HTTP gateway URL
IPFS_SERVICE_KEY	Shared secret key for ipfs-service authentication
MAX_BODY_SIZE	Maximum request body size for IPFS service (default: 5mb)

11. Testing Requirements

The project includes a comprehensive end-to-end test script (`test_full_journey_v2.sh`) that validates the complete patient journey:

- IPFS service health and CID operations (pin, fetch, unpin).
- Patient registration and EHR initialisation.
- Login for all roles across all organisations.
- Visit open, doctor assignment, vital recording, diagnosis update.
- Prescription creation and pharmacist dispensing.
- Lab request creation, acknowledgement, result submission, approval and return.
- Insurance claim submission, audit, and processing (APPROVED/REJECTED).
- Patient self-service EHR retrieval.
- Visit history and audit trail verification.
- Visit closure and medical record archival.

Test script supports flags: `--patient-id`, `--skip-ipfs`, `--stop-on-fail`.